

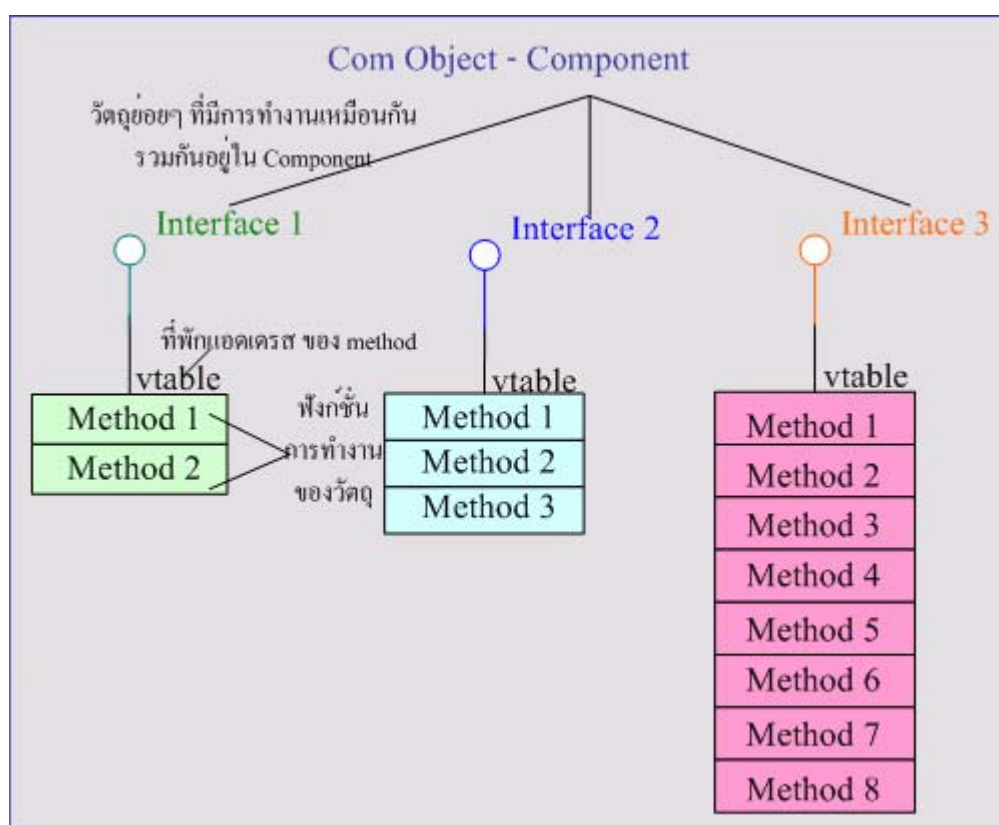
## บทที่ 2

### ทฤษฎีและหลักการคอมโพเน้นท์กับไดเรกซ์เอ็คซ์

#### 2.1 คอมโพเน้นท์ออบเจกต์โมเดล (Component Object Model – COM)

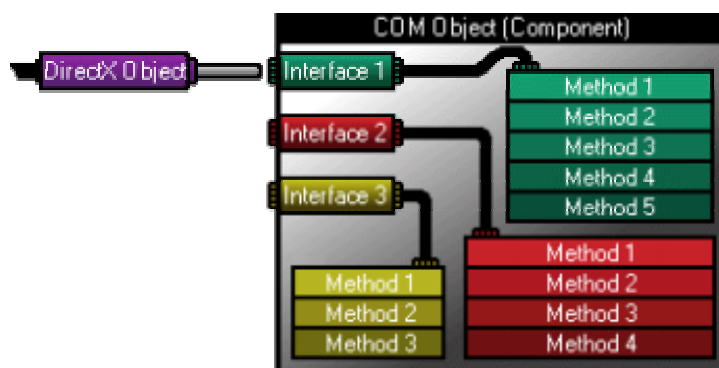
หลักการคอมโพเน้นท์ออบเจกต์โมเดลนี้ เป็นวิธีที่ทางไมโครซอฟท์ คิดขึ้นมาเพื่อให้ วัตถุต่างๆ ที่มีความเกี่ยวข้องหรือไม่เกี่ยวข้องกันก็ตาม สามารถติดต่อแลกเปลี่ยนข้อมูลกันได้ และสามารถนำส่วนการใช้งานมาใช้ใหม่ในโปรแกรมที่จะเขียนขึ้นใหม่ได้ โดยไม่ขึ้นกับภาษาที่ใช้เขียน ไม่ขึ้นกับระบบปฏิบัติการ แนวคิดวิธีที่จะทำให้ได้ผลดังกล่าว คือ ใช้หลักการออกแบบ วัตถุ (Object) ให้เป็นคอมโพเน้นท์ คือเป็นการรวมเอาวัตถุ (Object) ย่อยๆ ต่างๆ ที่มีหน้าที่การทำงานในแบบเดียวกัน มารวมกันเป็นคอมโพเน้นท์ โดยเรียก วัตถุ (Object) ย่อยๆ เหล่านั้นใหม่ว่าอินเตอร์เฟส (Interface)

คอมโพเน้นท์นั้น จะเป็นชุดที่รวมเอาวัตถุย่อยๆ หรืออินเตอร์เฟส ที่มีฟังก์ชันการทำงานในแบบเดียวกัน และในแต่ละอินเตอร์เฟสนั้น จะมีดีไวซ์ตัวหนึ่งที่มีชื่อว่า วิเทเบิล (vtable) สำหรับพักแอดเดรสของฟังก์ชัน (Method) ตามในรูปที่ 1



รูปที่ 2-1 แสดงความสัมพันธ์ของคอมโพเน้นท์, อินเตอร์เฟสและฟังก์ชัน

## 2.2 ไดรเร็กส์เอ็ช็อบเจ็กต์ กับ คอมโพเน็นท์ออบเจ็กต์



รูปที่ 2-2 แสดงการติดต่อกันระหว่าง COM ออบเจ็กต์ กับ ไดรเร็กส์เอ็ช็อบเจ็กต์

คอมโพเน็นท์ เปรียบเสมือน กล่องที่มีรูหรือช่องไว้ให้เสียบ ซึ่งอาจจะมีหลายๆ รู แต่ละรู เปรียบเสมือนอินเตอร์เฟส โดยแต่ละอินเตอร์เฟส ก็จะมีฟังก์ชันการทำงาน (Method) สำหรับไดเร็กส์เอ็ช็อบเจ็กต์ (DirectX Object) นั้น ก็เปรียบเสมือนแจ็ก ที่สามารถเสียบที่รูหรืออินเตอร์เฟสได้ หากวัตถุของไดเร็กส์เอ็ช็อบเจ็กต์ เสียบเข้าที่อินเตอร์เฟสใด ก็สามารถเรียกใช้งานฟังก์ชันการทำงานในอินเตอร์เฟสนั้นๆ ได้ หากต้องการใช้งานฟังก์ชันการทำงานอื่น ที่อยู่ในอินเตอร์เฟสอื่นๆ ก็ต้องสร้างไดเร็กส์เอ็ช็อบเจ็กต์ขึ้นมาใหม่ เพื่อนำไปเสียบกับอินเตอร์เฟสที่จะใช้งานฟังก์ชันการทำงานที่ต้องการ และถ้าเลิกใช้อินเตอร์เฟสใด ก็ต้องถอดแจ็กหรือวัตถุนั้นออก

กล่าวได้ว่า อินเตอร์เฟสแต่ละอัน ก็จะมีฟังก์ชันการทำงานต่างๆ ของตัวเอง หากต้องการจะใช้งานฟังก์ชันการทำงานได้ จะต้องสร้างไดเร็กส์เอ็ช็อบเจ็กต์ที่เข้าไปยังอินเตอร์เฟสที่มีฟังก์ชันการทำงานที่ต้องการ โดยการสร้างไดเร็กส์เอ็ช็อบเจ็กต์นี้ สามารถทำได้โดยการเรียกใช้ฟังก์ชัน ที่ ไดรเร็กส์เอ็ช็อบเจ็กต์เตรียมไว้ให้

การปรับปรุงความสามารถของวัตถุประเภท COM จะใช้การสร้างอินเตอร์เฟสตัวใหม่ขึ้นมา เพื่อใช้รองรับความสามารถใหม่มากกว่าจะเปลี่ยนแปลงฟังก์ชันที่มีอยู่ในอินเตอร์เฟสปัจจุบัน

## 2.3 อินเตอร์เฟสของ COM ออบเจ็กต์

ทุก COM ออบเจ็กต์ จะสนับสนุนอินเตอร์เฟสที่ชื่อ IUnknown อินเตอร์เฟสนี้เป็นอินเตอร์เฟสมาตรฐาน ซึ่งคอยควบคุมความสามารถในการใช้งานของไดเร็กส์เอ็ช็อบเจ็กต์ ที่เข้ามาใช้งานอินเตอร์เฟสต่างๆ โดยทุกอินเตอร์เฟสจะสืบทอดมาจากคลาส IUnknown ซึ่งเป็นคลาสที่ใช้สร้าง IUnknown อินเตอร์เฟส

สำหรับ IUnknown อินเตอร์เฟสนี้จะมีเมธอดอยู่ 3 ตัวด้วยกันคือ AddRef(), QueryInterface() และ Release() ดังนั้นอินเตอร์เฟสตัวอื่นๆ ที่สืบทอดไปจึงมีเมธอดทั้ง 3 นี้ด้วยเสมอ สังเกตได้จาก ไฟล์ Header สำหรับใช้งานอินเตอร์เฟส เช่น C:\DXSDK\include\dinput.h ในบรรทัดที่มีการใช้ #define

INTERFACE จะมีเมธอดทั้ง 3 ที่สืบทอดมาจาก IUnknown อยู่บรรทัดแรกๆ ซึ่ง COM ออบเจ็กต์ จะใช้ IUnknown เป็นตัวจัดการไคลเอนต์เอ็กซ์ออบเจ็กต์ที่เข้ามาใช้อินเตอร์เฟส ที่อยู่ใน COM ออบเจ็กต์นั้นๆ

- AddRef() จะทำการเพิ่มค่าการนับแก้ reference count ของออบเจ็กต์ ไปอีก 1 เมื่ออินเตอร์เฟส หรือ โปรแกรมประยุกต์อื่นๆ เข้าใช้งานออบเจ็กต์นั้นๆ

- QueryInterface() ตรวจสอบว่าออบเจ็กต์นั้นมีอินเตอร์เฟสที่ต้องการหรือไม่ และยังสามารถใช้สร้างไคลเอนต์เอ็กซ์ออบเจ็กต์ตัวใหม่ จากไคลเอนต์เอ็กซ์ออบเจ็กต์เดิมที่มีอยู่ได้ด้วย โดยจะไปสอบถาม COM ออบเจ็กต์ ซึ่งเป็นเจ้าของอินเตอร์เฟสที่ไคลเอนต์เอ็กซ์ออบเจ็กต์ตัวเดิมกำลังใช้อยู่ว่า มีอินเตอร์เฟสที่ไคลเอนต์เอ็กซ์ออบเจ็กต์ตัวใหม่ต้องการหรือไม่

- Release() ลดค่าการนับแก้ reference count ของออบเจ็กต์ ลงไป 1 เมื่อลดลงไปถึง 0 ออบเจ็กต์จะถูกทำลาย

ทุกครั้งที่สร้างไคลเอนต์เอ็กซ์ออบเจ็กต์ ซึ่งก็คือ การต้องการจะติดต่อกับอินเตอร์เฟสตัวใดในคอมโพเน้นท์ของไคลเอนต์เอ็กซ์ โดย COM ออบเจ็กต์ จะเรียก QueryInterface() เพื่อตรวจสอบว่ามีอินเตอร์เฟสที่ต้องการใช้มีอยู่ในออบเจ็กต์หรือไม่ หากมี จะเตรียมพื้นที่ในหน่วยความจำแล้วส่งตำแหน่งแอดเดรสกลับมา พร้อมทั้งเรียกใช้ AddRef() เพื่อเพิ่มการนับแก้ Reference Count อีก 1 ถ้ามีไคลเอนต์เอ็กซ์ออบเจ็กต์ตัวอื่นๆ มาใช้อินเตอร์เฟสอื่นๆ หรือใช้อินเตอร์เฟสซ้ำกันกับไคลเอนต์เอ็กซ์ออบเจ็กต์ตัวอื่น Reference Count ก็จะถูกรวบรวมค่าเพิ่มขึ้นไปเรื่อยๆ

โดยหน่วยความจำที่ไคลเอนต์เอ็กซ์ออบเจ็กต์ใช้นี้จะยังคงอยู่ตลอด หากไม่ทำการเรียกใช้ Release() แม้จะปิดโปรแกรมไปแล้วก็ตาม ซึ่งทำให้เกิดปัญหาเรื่องทรัพยากรตอบสนองไม่เพียงพอ หรือ Low resource ดังนั้น จะต้องเรียกใช้ Release() เมื่อไม่ต้องการใช้งานแล้วเสมอ เพื่อเป็นการคืนหน่วยความจำ และลดค่า Referenc Count ที่ละ 1 เมื่อลดจนเหลือ 0 ก็แสดงว่า ไม่มีไคลเอนต์เอ็กซ์ออบเจ็กต์ตัวใด ใช้ COM ออบเจ็กต์นั้นแล้ว กลไกภายใน COM ก็จะลบ COM ออบเจ็กต์ ตัวนั้นออกไปจากหน่วยความจำโดยอัตโนมัติ

## 2.4 ไคลเอนต์เอ็กซ์ 8 (DirectX 8)

### 2.4.1 ความรู้เบื้องต้นเกี่ยวกับไคลเอนต์เอ็กซ์

ไมโครซอฟท์ ไคลเอนต์เอ็กซ์ (Microsoft DirectX) เป็นชุดของแอปพลิเคชันโปรแกรมมิ่งอินเตอร์เฟส (Application Programming Interface – API) ได้รับการพัฒนาและออกแบบเพื่อใช้จัดเตรียมอินเตอร์เฟส สำหรับควบคุมฮาร์ดแวร์ทางด้านมัลติมีเดีย บนระบบปฏิบัติการ ไมโครซอฟท์ วินโดวส์ (Microsoft Windows) ได้อย่างมีประสิทธิภาพ และเป็นเครื่องมือให้โปรแกรมเมอร์ใช้คำสั่งควบคุมฮาร์ดแวร์ได้อย่างใกล้ชิด โดยไม่ต้องยุ่งยากกับการสร้างส่วนโปรแกรมติดต่อในระดับล่าง ซึ่งเครื่องมือที่ใช้ติดต่อจะแตกต่างกันไปตามประเภทของอุปกรณ์ ทำให้โปรแกรมเมอร์สร้างซอฟต์แวร์เพื่อทำงานทางด้านมัลติมีเดียได้เป็นอย่างดี เนื่องจากการติดต่อในแต่ละคอมโพเน้นท์จะเป็นของอุปกรณ์ประเภทต่างๆ เป็นอิสระในการเขียนโปรแกรม

API ของ ไดรฟ์เอ็กซ์นั้น สร้างขึ้นมาจากฐานของ HAL (Hardware Abstraction Layer) สามารถซ่อนลักษณะของดีไวซ์ (Device) ที่เกี่ยวข้องกับฮาร์ดแวร์ และออกแบบมาเพื่อให้สามารถรองรับความขยายตัวของฮาร์ดแวร์ใหม่ๆ ที่จะออกมาในอนาคต ดังนั้นจึงสามารถรองรับความสามารถของฮาร์ดแวร์ที่มีความสามารถเร่งความเร็วใหม่ๆ ที่ปัจจุบันยังไม่มี ด้วยความสามารถในการจำลองการทำงานผ่าน HEL (Hardware Emulation Layer) หรือเลือกที่จะหลีกเลี่ยงไม่ใช้ความสามารถนี้ ถ้า HEL ไม่สนับสนุน ทำให้โปรแกรมเมอร์พัฒนางานทางด้านมัลติมีเดียง่ายขึ้น ไม่จำเป็นต้องทดสอบกับอุปกรณ์ทุกตัวในท้องตลาด เพียงแต่ทดสอบกับไดรฟ์เอ็กซ์ก็เพียงพอแล้ว สำหรับผู้สร้างอุปกรณ์ฮาร์ดแวร์ ก็ต้องสร้างไดรฟ์เอ็กซ์ของอุปกรณ์นั้นๆ เพื่อให้สามารถทำงานเข้ากับไดรฟ์เอ็กซ์ได้

#### 2.4.2 ชุดพัฒนาซอฟต์แวร์สำหรับไดรฟ์เอ็กซ์ (DirectX software Development Kit : DirectX SDK)

ชุดพัฒนาซอฟต์แวร์สำหรับไดรฟ์เอ็กซ์ นี้ประกอบด้วยชุดของไลบรารี, ไฟล์ DLL, ไฟล์ header รวมไปถึงเอกสาร และตัวอย่างโปรแกรม ซึ่งช่วยในการพัฒนาซอฟต์แวร์ทางด้านมัลติมีเดีย ซึ่งการใช้ไดรฟ์เอ็กซ์ได้นั้น ก็ต้องทำการลงชุดพัฒนาซอฟต์แวร์สำหรับไดรฟ์เอ็กซ์นี้ก่อน

#### 2.4.3 คอมโพเน้นท์ในไดรฟ์เอ็กซ์ 8

ไดรฟ์เอ็กซ์ 8 (DirectX 8) จะประกอบด้วย Component 6 ตัว คือ

##### 2.4.3.1 DirectX Graphics

DirectX Graphics เป็นคอมโพเน้นท์ใหม่ที่รวมคอมโพเน้นท์เดิมที่ชื่อ DirectDraw กับ Direct3D ไปเป็นอินเตอร์เฟสอันเดียวคือ Direct3D Interface ซึ่งอยู่ในคอมโพเน้นท์ DirectX Graphics นี้ ใช้จัดการทางด้านกราฟฟิกการแสดงผลภาพ 2 มิติ และวัตถุ 3 มิติ ซึ่งช่วยจัดการการใช้อุปกรณ์แสดงผล 3 มิติ ในรูปแบบที่ไม่ขึ้นกับชนิดของอุปกรณ์ที่จะใช้ (Device-Independent)

##### 2.4.3.2 DirectX Audio

DirectX Audio เป็นคอมโพเน้นท์ใหม่ที่รวมคอมโพเน้นท์เดิมที่ชื่อ DirectSound กับ DirectMusic มาเป็นอินเตอร์เฟสในคอมโพเน้นท์ Direct X Audio นี้ ใช้จัดการทางด้านเสียง สนับสนุนการเร่งความเร็วทางด้านฮาร์ดแวร์ (Hardware acceleration) ในการสร้างเสียงแบบไดนามิก (Dynamic soundtrack) และเอฟเฟ็คเสียง 3 มิติขั้นสูง (Advanced 3D position effect)

##### 2.4.3.3 DirectInput

DirectInput ใช้จัดการทางด้านการรับข้อมูลจากอุปกรณ์อินพุต เช่นจาก คีย์บอร์ด, เมาส์, จอยสติ๊ก เป็นต้น เพื่อเข้าถึงข้อมูลด้วยการติดต่อโดยตรงกับฮาร์ดแวร์ด้วยไดรฟ์เอ็กซ์ไดรฟ์เอ็กซ์ (Hardware Driver) ซึ่งจะทำงานได้เร็วกว่าการติดต่อด้วยเมสเสจของวินโดวส์ นอกจากนี้ยังสนับสนุนการทำงานกับอุปกรณ์แบบแรงตอบสนอง (Force Feedback) อีกด้วย

#### 2.4.3.4 DirectPlay

DirectPlay เป็นมีเดียที่เป็นอิสระบนเน็ตเวิร์ก ซึ่งหมายถึง สามารถรันได้บนเครือข่ายแบบ TCP/IP, เครือข่ายแบบ IPX หรือแม้แต่การต่อตรงจากโมเด็ม และการเป็นอิสระทางเน็ตเวิร์กนี้จะสามารถถูกพัฒนาต่อไปได้ในอนาคต นั่นคือ สามารถรองรับอุปกรณ์และโปรโตคอลใหม่ๆ เช่น IPV6, multicast โดยปกติ DirectPlay ใช้จัดการการทำงานด้านเน็ตเวิร์ก โดยไม่จำเป็นต้องไปยุ่งกับการทำงานในระดับล่าง มีหน้าที่การทำงานสำหรับการส่งข้อมูลที่เดียวให้หลายๆเครื่องเป็นกรุปได้ สามารถเขียนโปรแกรมติดต่อทางด้านเน็ตเวิร์ก ทั้งในการติดต่อผ่านเน็ตเวิร์กระหว่างโปรแกรมแบบมัลติเพลเยอร์ (Multiplayer) แบบเพียร์ทูเพียร์ (Peer-to-Peer) และแบบไคลเอ็นท์/เซิร์ฟเวอร์ (Client/Server)

#### 2.4.3.5 DirectShow

DirectShow ใช้สำหรับการทำงานด้านมัลติมีเดีย เป็นสถาปัตยกรรมทางด้านมีเดียสตรีมมิ่ง (media-streaming) ของไมโครซอฟท์ วินโดวส์ ที่มีความสามารถในการแคปเจอร์ (Capture) สูง สามารถบรรจุข้อมูลที่เป็นวิดีโอ และการบีบอัดข้อมูลไฟล์ (Audio data compressed) ได้หลายรูปแบบ เช่น การเล่นไฟล์วิดีโอ ประเภท MPEG, Audio-Video(AVI), MPEG-1 Layer3(MP3) และไฟล์นามสกุล .wav เป็นต้น

#### 2.4.3.6 DirectSetup

DirectSetup ใช้สำหรับการจัดการทางด้านการติดตั้งองค์ประกอบที่จะต้องใช้ในงานไคลเอนต์ เอ็กซ์บนเครื่องคอมพิวเตอร์ของผู้ใช้อย่างอัตโนมัติ หากเครื่องคอมพิวเตอร์นั้นยังไม่ได้ทำการติดตั้ง